从 QEMU 复位到 Krunix 的第一条指令

以 RustSBI-QEMU 源码、LLDB 反汇编和本机执行跟踪为线索，理解固件怎样准备启动参数、进入陷入处理，再用 `mret` 交给内核。

`0x1000` QEMU 复位 ROM → `0x80000000` RustSBI · M 模式 → `0x80200000` Krunix · S 模式

> **源码和镜像的对应范围**
>
> 你贴出的 `main.rs` 横幅写的是 SBI v2.0.0。本机镜像 `bootloader/rustsbi-qemu.bin` 打印 RustSBI 0.3.1、SBI v1.0.0。本文用源码解释函数职责，用本机镜像和 QEMU 执行跟踪核对具体地址及启动分支。

复位 ROM 如何进入 RustSBI

地址	指令	执行结果
`0x1000`	`auipc t0, 0`	`t0 = 0x1000`，作为后续数据访问的基址。
`0x1004`	`addi a2, t0, 0x28`	`a2` 指向 `0x1028` 的动态固件信息。所读 RustSBI 入口接收 `a0` 和 `a1`。
`0x1008`	`csrr a0, mhartid`	`a0` 保存当前硬件线程 ID，单 hart 配置下为 0。
`0x100c`	`ld a1, 0x20(t0)`	读出 `0x1020` 里存的设备树地址，放入 `a1`。
`0x1010`	`ld t0, 0x18(t0)`	读出 `0x1018` 里存的固件入口 `0x80000000`。
`0x1014`	`jr t0`	跳到 `0x80000000`，开始执行固件。

在复位处用 `x/2gx 0x1018` 可以一次读取入口地址和设备树地址两个 64 位值。你此前的日志中，设备树位于 `0x87e00000`。

QEMU 复位向量生成代码 · RustSBI-QEMU main.rs

固件入口与平台初始化

`#![no_std]` 和 `#![no_main]` 让固件自行提供入口。`_start` 是放在 `.text.entry` 的裸函数，由其中的汇编建立栈和安排调用。

```
0x80000000  auipc ra, 0x2
0x80000004  jalr  0x342(ra)   → 0x80002342  trap_stack::locate
0x80000008  auipc ra, 0x0
0x8000000c  jalr  0x74(ra)    → 0x8000007c  RustSBI rust_main
0x80000010  j     0x80001690  → trap_vec
```

第一次调用先用 `auipc` 算出 `0x80002000`，`jalr` 再加 `0x342`，并把返回地址 `0x80000008` 写入 `ra`。栈定位函数按 `mhartid` 选择每 hart 16 KiB 的栈区。它的收尾在 `0x80002d72` 预留 48 字节，随后 `andi sp, sp, -0x8` 将栈顶向下对齐到 8 字节。`0x80002d78` 的 `ret` 返回入口，接着调用固件的 `rust_main`。

只有第一个 hart 执行的部分

`GENESIS.swap(false, Ordering::AcqRel)` 选出执行全局初始化的 hart。它先清零固件的 `.bss`，再用 `opaque` 指向的设备树建立 `BOARD_INFO`。其他 hart 等待设备信息发布，然后各自设置 PMP 和陷入栈。

初始化对象	它解决什么
UART 与控制台	从设备树找到串口地址，注册固件输出，打印平台名、内存范围、固件地址和内核入口。
CLINT、QEMU Test、DBCN	分别提供计时器和软件中断、关机或复位，以及 SBI 调试控制台服务。
`FixedRustSBI`	用 `#[derive(RustSBI)]` 把 IPI、计时器、hart 管理、复位和 DBCN 接到统一的 SBI 分发入口。
PMP	限制 S 模式访问固件占用区 `0x80000000..0x80200000`，允许 S 模式对从 `0x80200000` 开始的主内存读、写、执行。
`mideleg`、`medeleg`、`mtvec`	配置可委托的中断和异常，并登记 M 模式陷入向量。S 模式的 `ecall` 留给 M 模式处理。

固件把 `Supervisor { start_addr: 0x80200000, opaque }` 登记到本 hart 的 HSM。之后 `rust_main` 返回，入口代码跳向 `trap_vec`，由后续处理逻辑实际启动内核。

RustSBI-QEMU 的 _start、rust_main 与 set_pmp

陷入入口如何换栈、保存现场

首次启动时，固件从 `0x80000010` 主动跳到 `trap_vec`，再从 `0x80001690` 跳到公共入口 `0x80002d7c`。这解释了为什么 `0x80002d78` 的 `ret` 和后面的 `csrrw` 虽然相邻，实际执行时却分属两段路径。

反汇编位置	状态如何变化
`0x80002d7c`	`csrrw sp, mscratch, sp` 交换两者。`sp` 得到固件陷入栈指针，`mscratch` 暂存进入前的 `sp`。
`0x80002d80` – `0x80002d82`	先把原 `a0` 存到 `sp + 0x10`，再从 `sp + 0x0` 取出寄存器上下文指针，放进 `a0`。
`0x80002d84` – `0x80002da0`	把 `ra` 和 `t0..t6` 存入上下文。后面的函数调用可以改写这些寄存器。
`0x80002da4` – `0x80002da8`	`mv a0, sp` 把陷入栈指针作为参数传给回调。`ld ra, 0x8(sp)` 取函数地址，随后 `jalr ra` 调用它。本次执行跟踪的目标是 `0x8000061e`。
`0x80002daa` 起	把上下文指针重新加载到 `a1`，用 `a0` 里的回调返回值决定如何恢复寄存器。

连续的 `addi -1` 和 `beqz` 在比较什么

每减一次 1 再判断是否为零，就检查了下一个原始返回值。下表用 `FastResult` 文档的名称对照本机反汇编。

原返回值	分支入口	寄存器动作
`0` · FastCall	`0x80002e4a`	加载目标上下文的 `a0` 和 `a1`。
`1` · Call	`0x80002e3a`	再多加载 `a2..a7`。
`2` · Restore	`0x80002e1a`	再多恢复 `ra` 和 `t0..t6`。
`3` · Switch	`0x80002dee`	额外恢复 `s0..s11`，然后继续执行下方的恢复代码。
`4` · Continue	`0x80002dba`	额外保存 `s0..s11`，调用完整处理函数后再分发。

fast-trap 陷入栈设计 · FastResult 枚举

为什么一条 `mret` 就进入了内核

首次启动时，`fast_handler` 从 HSM 取到待启动的 `Supervisor`。它为 S 模式准备返回状态，将 `mstatus.MPP` 设为 S，关闭 S 模式中断并清零 `satp`。它还把 hart ID、设备树地址和内核入口分别放进目标上下文的 `a0`、`a1` 和 PC。

源码里的 `ctx.call(2)` 选择只传两个参数的快速路径。本机执行跟踪中，回调返回 `0`，`beqz` 因此跳到 `0x80002e4a`，跳过保存 `s0..s11` 的完整路径。

```
0x80000864   固件处理函数写入 mepc
0x80002dac   a0 == 0，跳到 0x80002e4a
0x80002e4a   从目标上下文加载 a0 和 a1
0x80002e4e   交换 sp 与 mscratch
0x80002e52   mret
0x80200000   Krunix _start 的第一条指令
```

> **两个返回指令的区别**
>
> `ret` 用 `ra` 作为目标地址，完成普通函数返回。`mret` 用 `mepc` 作为目标地址，并根据 `mstatus.MPP` 恢复特权模式。固件在执行 `mret` 前已经准备好地址和模式，因此下一条指令就在 `0x80200000` 的 S 模式执行。

处理函数可以在最终交换前写入 `mscratch`。本次跟踪在 `0x80000860` 观察到了这次写入，所以最后一次 `csrrw` 会切入目标上下文。Krunix 的 `_start` 紧接着执行 `la sp, boot_stack_top`，设置自己的启动栈，再调用内核的 `rust_main`。

内核运行后怎样再次进入 RustSBI

Krunix 的 `rust_main` 清除自己的 `.bss`，初始化 logger，输出内存布局和 `Hello World`。字符输出通过 `sbi_rt::legacy::console_putchar` 发起 SBI 调用，由固件向 UART 写字符。关机则调用 SBI 的系统复位接口。

标准 SBI 调用用 `a7` 传扩展 ID、`a6` 传函数 ID、`a0..a5` 传参数。S 模式执行 `ecall` 后，固件识别 `SupervisorEnvCall`，交给 `handle_ecall`，再把返回值写进 `a0`、`a1`。它推进 `mepc`，使内核返回后从 `ecall` 的下一条指令继续。

hart 管理与异常处理 · `hart_start` 给目标 hart 登记入口和参数，再通过 CLINT 软件中断唤醒它。停止的 hart 在处理循环里用 `wfi` 等待启动请求。`hart_get_status` 查询状态，后端的 `hart_suspend` 支持保留上下文的挂起。遇到未支持的陷入时，固件打印陷入原因及 `mstatus`、`mepc`、`mtval` 并进入 panic，随后请求 QEMU 关机。

RustSBI-QEMU 的 fast_handler · RISC-V 特权架构 · SBI 调用约定

用 LLDB 亲手跟一遍

在 VS Code 选择 **Krunix · QEMU (LLDB)** 并按 F5。当前项目配置会让 QEMU 暂停在 `0x1000`。在 VS Code 调试控制台输入命令时，行首加一个反引号。直接使用 LLDB 终端时按下面的写法输入。

```
x/10i $pc
x/2gx 0x1018
breakpoint set --address 0x80002d7c
breakpoint set --address 0x80002e52
breakpoint set --address 0x80200000
continue
```

停住的位置	看什么	怎么验证
`0x1000`	QEMU 的复位 ROM 和它使用的两个地址。	`x/10i $pc` 看汇编，`x/2gx 0x1018` 读固件入口和 DTB 地址。
`0x80002d7c`	进入固件陷入栈时 `sp` 的变化。	先用 `register read sp` 记录栈顶，执行一次 `si` 后再读一次。
`0x80002e52`	`mret` 交出控制权。	用 `x/1i $pc` 确认当前指令。执行 `si` 后，用 `register read pc` 查看目标地址。
`0x80200000`	Krunix 设置启动栈并调用内核入口。	`x/6i $pc` 显示 `la` 与 `call` 在 ELF 中对应的机器指令。

相关源码入口

- QEMU · riscv_setup_rom_reset_vec，复位 ROM 指令和两项内嵌地址。
- RustSBI-QEMU · main.rs，重点看 `_start`、`rust_main`、`fast_handler` 和 `Hsm`。
- fast-trap · 陷入栈与快速路径，解释为什么要换栈，以及何时保存更多寄存器。
- Krunix 仓库 · `src/arch/riscv64/entry.asm`、`src/main.rs`、`src/sbi.rs`，分别对应启动汇编、内核初始化和 SBI 调用。

> 对照单步时，先读 `$pc`，再区分指令里的**地址**、**该地址存的值**和**跳转目标**。复位 ROM 的 `ld`、固件入口的 `jalr`、最后的 `mret` 分别展示了三种不同的取值和控制转移。